

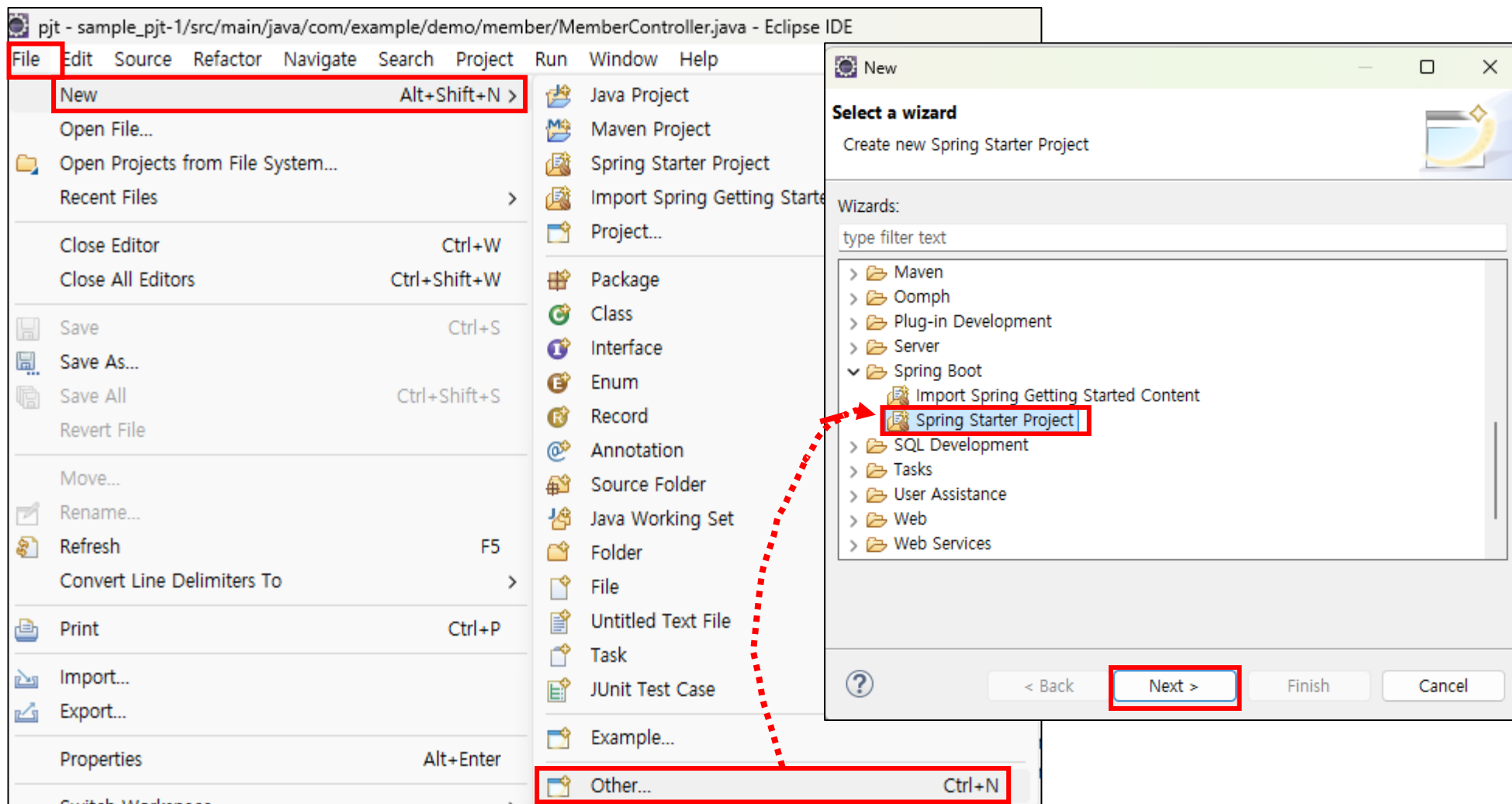
5강

spring 프로젝트 개발





➤ Spring boot 프로젝트 생성 하기 - 1





➤ Spring boot 프로젝트 생성 하기 - 2

New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

Package:

Working sets

☐ Add project to working sets

Working sets:



[Project Metadata]

Group: 프로젝트의 그룹 ID로, 보통 조직의 도메인 이름을 역순으로 입력합니다.(예. com.office나 org.springframework)

Artifact: 프로젝트의 아티팩트(산출물) ID로, 생성될 JAR 파일의 이름을 정의합니다.(예. ch03_pjt_01, ourproject)

Name: 프로젝트 이름으로, 기본적으로 아티팩트와 동일한 값을 가집니다. 이 값은 build.gradle에서 프로젝트 이름으로 사용됩니다.

Package name: 패키지 이름을 설정할 수 있는 옵션으로, 보통 group + artifact 형식으로 설정됩니다.

[Project]

Maven Project 또는 Gradle Project를 선택할 수 있습니다.

Maven: XML 기반의 pom.xml 파일을 사용해서 프로젝트 의존성, 빌드, 배포 등을 관리합니다. 주로 Spring Legacy에서 사용합니다.

Gradle: 빠르고 유연한 빌드 시스템을 제공하는 빌드 도구로 build.gradle 파일을 사용합니다. 주로 Spring Boot에서 사용합니다.

[Language]

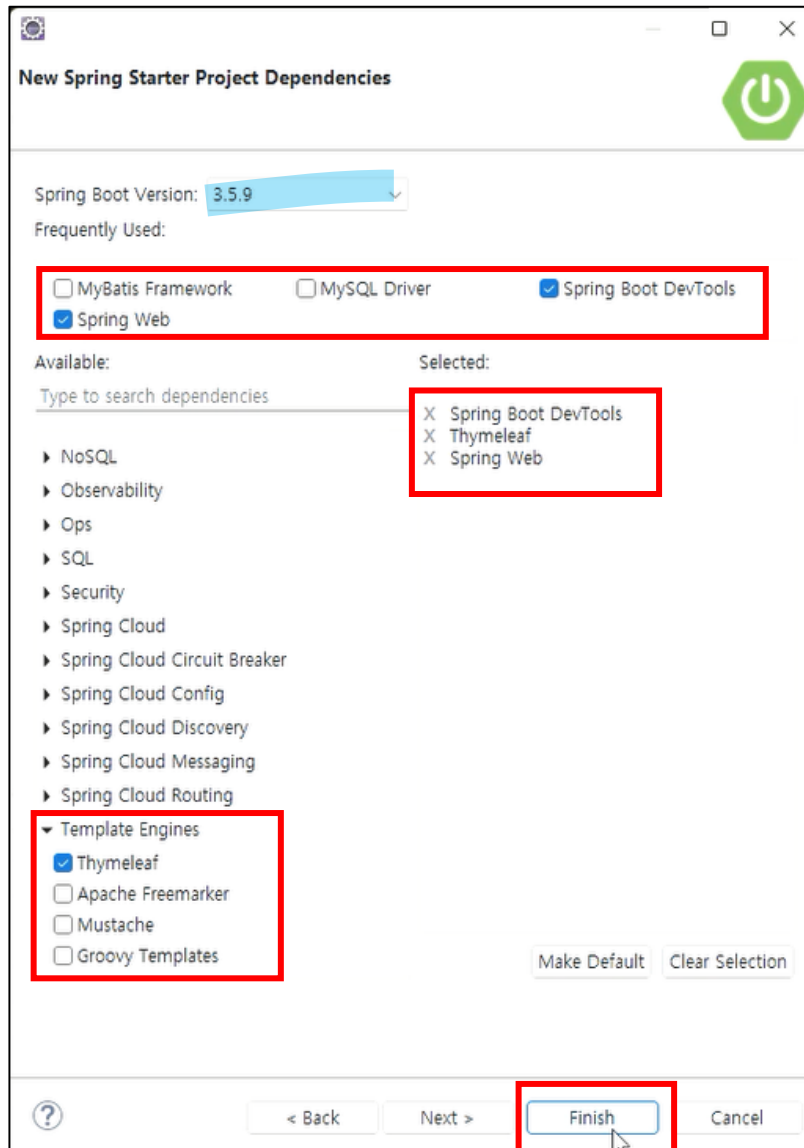
개발언어를 Java, Kotlin, Groovy 중에서 선택할 수 있습니다.

[Spring Boot]

Spring Boot의 버전을 선택합니다. 가능한 보안 패치와 성능 개선이 포함되어 있는 최신 버전을 선택하는 것이 좋습니다.

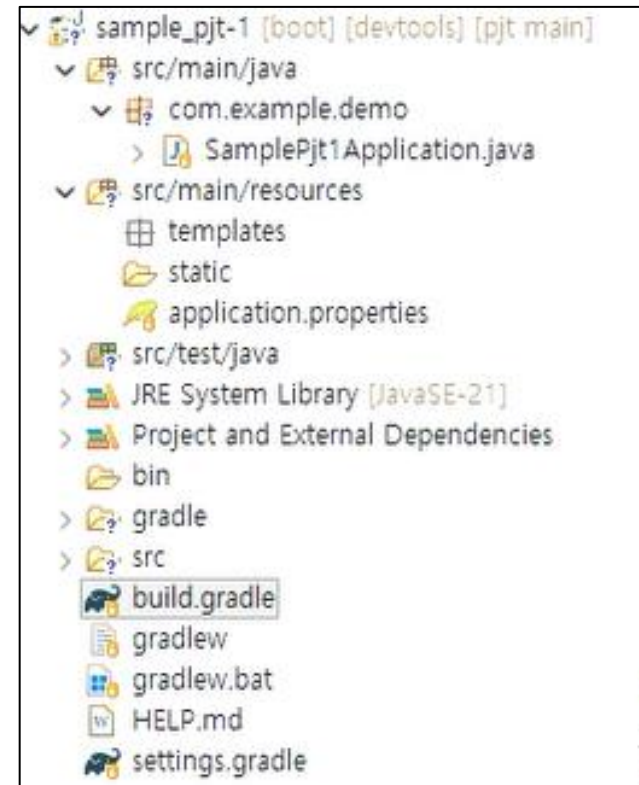


➤ Spring boot 프로젝트 생성 하기 - 3



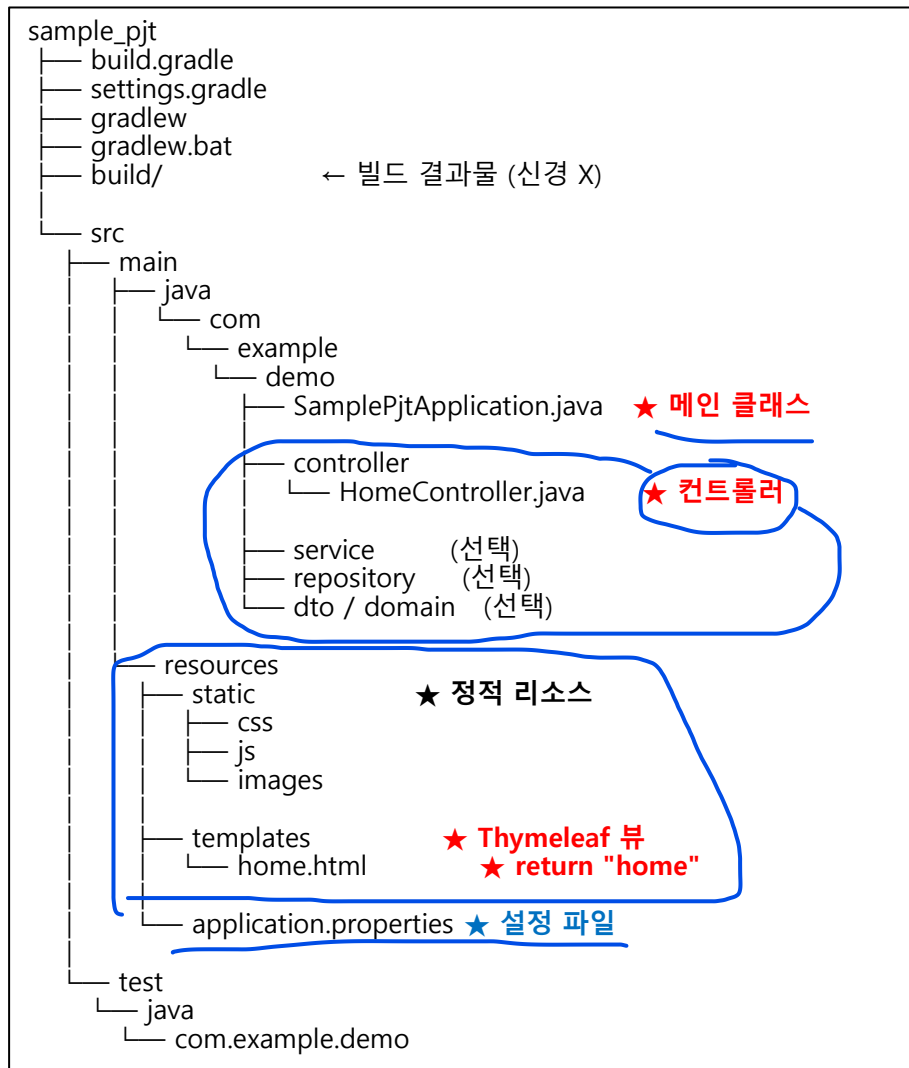
✓ 기본적으로 필요한 의존 모듈을 설정

- spring web 모듈
- spring boot dev tools 모듈
- thymeleaf 모듈





➤ Spring boot 프로젝트 디렉토리 기본 구조



1) java 디렉터리

- Java 파일이 위치하며, Spring Boot 애플리케이션의 컨트롤러, 서비스 등의 핵심 로직을 구현합니다.

2) resources 디렉터리

- resources에는 애플리케이션 설정, 템플릿, 정적 파일 등이 위치합니다.
- static: 정적 리소스 (예: HTML, CSS, JavaScript, 이미지 등)파일이 위치합니다.
- templates: 뷰(view) 파일이 위치합니다

3) application.properties 파일

- 애플리케이션 설정 변수를 정의하는 파일로, 애플리케이션의 동작을 구성하는 다양한 속성들을 설정하는 데 사용됩니다.

spring.application.name=ch03_pjt_01

애플리케이션의 이름을 지정합니다.

Tomcat

server.port=8090

Spring Boot 애플리케이션의 HTTP 서버 포트를 설정합니다.

기본적으로 Spring Boot는 8080 포트에서 실행되지만, 8090 포트에서 실행되도록 합니다.



➤ Spring boot **application.properties** 파일(=환경 설정)

- 단, properties 환경설정 파일 수정 후 프로젝트 폴더 오른쪽 버튼 클릭 -> Gradle 클릭 -> Refresh Gradle Project 실행한다.

```
spring.application.name=sample_pjt-1
```

```
# Tomcat 서버를 내장되어 있다.
```

```
# 원래 포트번호는 8080인데 => 8090으로 변경함
```

```
# Tomcat
```

```
server.port=8090
```

```
# Dev Tools
```

```
spring.devtools.restart.enabled=true
```

```
# Thymeleaf #주석
```

```
spring.thymeleaf.cache=false
```

```
spring.thymeleaf.prefix=classpath:/templates/
```

```
spring.thymeleaf.suffix=.html
```

```
spring.thymeleaf.check-template-location=true
```



➤ Spring boot **application.properties** 파일(=환경 설정)

Dev Tools

spring.devtools.restart.enabled=true

Spring DevTools의 자동 리스타트 기능을 활성화합니다.

애플리케이션 코드가 변경될 때 자동으로 서버가 재시작되어 변경 사항이 즉시 반영됩니다

.

spring.thymeleaf.cache=false

Thymeleaf 템플릿 캐싱을 비활성화합니다.

개발 환경에서는 false, 운영 환경에서는 true(기본값) 로 설정하는 것이 일반적입니다.

spring.thymeleaf.prefix=classpath:/templates/

Thymeleaf 템플릿 파일의 위치를 설정합니다.

classpath:/templates/는 템플릿이 resources/templates 폴더에 위치하는 것을 의미합니다.

spring.thymeleaf.suffix=.html

템플릿 파일의 확장자를 .html로 정의합니다.

예) src/main/resources/templates/home.html과 같은 HTML 템플릿을 사용할 수 있습니다.

spring.thymeleaf.check-template-location=true

Thymeleaf가 템플릿 파일의 위치를 확인할지, 안 할지의 작업 여부를 설정합니다.

애플리케이션이 실행될 때, Thymeleaf 파일이 존재하는지 확인하고 없으면 경고 또는 오류를 발생 시킵니다. 개발 환경에서는 true로 설정해서 템플릿 누락을 방지합니다.



➤ Spring boot 프로젝트 Hello 간단 예제

- src -> main -> java -> HomeController 클래스 생성 후 아래와 같이 작성한다.

```
package com.example.demo;

import org.springframework.stereotype.Controller;

@Controller
public class HomeController {

    // Get방식으로 요청하면 "/"이 아이와 Mapping(연결)하겠다는 의미
    // https://localhost:8090/ => 이 요청이 들어오면
    // home()메소드가 호출된다.
    @GetMapping("/") // https://localhost:8090/ 홈지정하는 부분
    public String home() {
        return "home"; // home.html로 반환하는데 -> 반환하는 역할을 viewResolver가한다.
        // 그럼 어떻게 home.html에서 html확장자가 자동으로 생성되는 이유는
        // application.properties의 spring.thymeleaf.suffix=.html
        // => 이부분 꼬리말이 html로 환경이 설정되어 있기 때문에 가능한 것이다.
        // spring.thymeleaf.prefix=classpath:/templates/
        // => 이부분의 머릿말로 home.html이 templates폴더 안에 들어있음을 알려준다.
        // 이 모든 작업을 viewResolver가 담당한다.
    }
}
```



➤ Spring boot 프로젝트 Hello 간단 예제

- src -> main -> resources -> templates -> home.html 작성한다.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Insert title here</title>
</head>
<body>
  This page is HoME.
</body>
</html>
```

5강
spring 프로젝트
@ 어노테이션





➤ 스프링 부트에서 어노테이션의 역할

✓ 어노테이션이란?

“이 클래스/메서드는 이런 역할을 한다”라고 스프링에게 알려주는 표식(메타데이터)

```
@Component  
public class MemberService {  
}
```

“이 클래스는 스프링이 관리해야 할 객체(Bean) 입니다”

라고 설정 파일 없이 코드로 알려주는 것

스프링에게 일을 시키는 지시문 이다.

✓ 어노테이션을 쓰는 이유

이유	설명
설정 파일 감소	XML 설정 거의 제거
가독성 향상	코드만 봐도 역할이 보임
자동 처리	객체 생성, 주입, 실행을 <u>스프링이 알아서</u>
유지보수 용이	구조 변경 시 수정 포인트 감소



➤ 스프링 부트에서 어노테이션의 종류(영역별)

- 스프링 부트 기본 / 설정 어노테이션

어노테이션	역할
@SpringBootApplication	스프링 부트 시작 어노테이션 (아래 3개 포함)
@EnableAutoConfiguration	자동 설정 활성화
@ComponentScan	컴포넌트 자동 탐색
@Configuration	설정 클래스임을 명시
@Bean	개발자가 직접 Bean 등록
@Import	다른 설정 클래스 포함
@PropertySource	properties 파일 로드



➤ 스프링 부트에서 어노테이션의 종류(영역별)

- Bean 등록 (객체 생성 관련)



어노테이션	용도
@Component	일반 컴포넌트
@Service	서비스 로직
@Repository	DAO / DB 접근
@Controller	MVC 컨트롤러
@RestController	REST API 컨트롤러
@Configuration	설정 클래스

- 의존성 주입 (DI)

어노테이션	설명
@Autowired	자동 주입
@Qualifier	같은 타입 Bean 중 선택
@Primary	우선 주입 대상
@Resource	이름 기준 주입 (JSR)
@Inject	표준 DI (JSR-330)



➤ 스프링 부트에서 어노테이션의 종류(영역별)

- MVC / 웹 요청 처리 : 컨트롤러 관련

어노테이션	역할
@Controller	화면 반환
@RestController	JSON 반환
@RequestMapping	요청 URL 매핑
@GetMapping	GET 요청 — 공개
@PostMapping	POST 요청 — 비공개
@PutMapping	PUT 요청
@DeleteMapping	DELETE 요청



➤ 스프링 부트에서 어노테이션의 종류(영역별)

- 요청 데이터 처리

어노테이션	설명
<code>@RequestParam</code>	쿼리 파라미터
<code>@PathVariable</code>	URL 변수
<code>@RequestBody</code>	JSON → 객체
<code>@ResponseBody</code>	객체 → JSON
<code>@ModelAttribute</code>	폼 데이터 바인딩

- 예외 처리

어노테이션	설명
<code>@ExceptionHandler</code>	예외 처리 메서드 <i>try catch</i>
<code>@ControllerAdvice</code>	전역 예외 처리
<code>@RestControllerAdvice</code>	REST 전역 예외



➤ 스프링 부트에서 어노테이션의 종류(영역별)

- JPA / DB 관련

어노테이션	역할
@Entity	DB 테이블 매핑
@Table	테이블 이름 지정
@Id	PK
@GeneratedValue	자동 증가
@Column	컬럼 매핑
@ManyToOne	N:1 관계
@OneToMany	1:N 관계
@JoinColumn	FK 설정
@Transactional	트랜잭션 처리



➤ 스프링 부트에서 어노테이션의 종류(영역별)

- 검증 (Validation)

어노테이션	설명
@NotNull	null 불가
@NotEmpty	빈 값 불가
@NotBlank	공백 불가
@Size	길이 제한
@Email	이메일 형식
@Pattern	정규식

- 테스트 관련

어노테이션	설명
@SpringBootTest	통합 테스트
@Test	테스트 메서드
@MockBean	가짜 Bean
@WebMvcTest	컨트롤러 테스트



➤ 스프링 부트에서 어노테이션의 종류(영역별)

- 검증 (Validation)

어노테이션	설명
@NotNull	null 불가
@NotEmpty	빈 값 불가
@NotBlank	공백 불가
@Size	길이 제한
@Email	이메일 형식
@Pattern	정규식

- 테스트 관련

어노테이션	설명
@SpringBootTest	통합 테스트
@Test	테스트 메서드
@MockBean	가짜 Bean
@WebMvcTest	컨트롤러 테스트



➤ 스프링 부트에서 어노테이션의 종류(영역별)

- 기타 자주 쓰는 어노테이션

어노테이션	설명
@Value	설정 값 주입
@Profile	실행 환경 분기
@EnableScheduling	스케줄링
@Scheduled	주기 실행
@Async	비동기 처리
@EnableCaching	캐시 사용

- “어노테이션은 주석이 아니라 ‘명령어’다”
- “스프링에게 이 클래스가 뭐하는 애인지 알려주는 표식이다”
- “객체 생성, 연결, 실행을 우리가 안 하고 스프링이 하게 만든다”

@Component → 객체 만들어라

@Autowired → 연결해라

@Controller → 요청 받아라

@GetMapping → 이 URL 처리해라

@Transactional → 실패하면 롤백해라

5강

spring 프로젝트

회원 가입하기





➤ Spring boot **application.properties** 파일(=환경 설정)

- 단, properties 환경설정 파일 수정 후 프로젝트 폴더 오른쪽 버튼 클릭 -> Gradle 클릭 -> Refresh Gradle Project 실행한다.

```
spring.application.name=sample_pjt-1
```

```
# Tomcat 서버를 내장되어 있다.
```

```
# 원래 포트번호는 8080인데 => 8090으로 변경함
```

```
# Tomcat
```

```
server.port=8090
```

```
# Dev Tools
```

```
spring.devtools.restart.enabled=true
```

```
# Thymeleaf #주석
```

```
spring.thymeleaf.cache=false
```

```
spring.thymeleaf.prefix=classpath:/templates/
```

```
spring.thymeleaf.suffix=.html
```

```
spring.thymeleaf.check-template-location=true
```



➤ Spring boot 프로젝트 회원 가입 하기

- src -> main -> java -> MemberController 클래스 생성 후 아래와 같이 작성한다.

```
@Controller // Controller이라는 Bean객체가 생성되도록 어노테이션처리 한다.
public class MemberController {

    private final SamplePjt1Application samplePjt1Application;

    MemberController(SamplePjt1Application samplePjt1Application) {
        this.samplePjt1Application = samplePjt1Application;
    }

    // 아래는 핸들러 메소드들이다.
    //회원 가입 양식
    @GetMapping("/member/singup")
    public String singUpForm() {
        System.out.println("singUpForm()");
        return "singUpForm"; //응답에 사용하는 html이름
    }

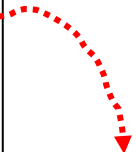
    //로그인 양식
    @GetMapping("/member/singin")
    public String singinForm() {
        System.out.println("singinForm()");
        return "singInForm2"; //응답에 사용하는 html이름
    }
}
```



➤ Spring boot 프로젝트 회원 가입 하기

- src -> main -> resources -> templates -> signUpForm.html 작성한다.

```
<!DOCTYPE html>
<html lang="kor" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <form th:action="@{/member/signup_confirm}" method="POST">
        <input type="text" name="id" placeholder="id" />
        <br>
        <input type="password" name="pw" placeholder="pw" />
        <br>
        <input type="text" name="email" placeholder="email" />
        <br>
        <input type="submit" value="SING-UP" />
    </form>
</body>
</html>
```



Insert title here

localhost:8090/member/signup

지마켓 쿠팡! G마켓 - 쇼핑을 바... Gmail YouTube

id

pw

email

SING-UP



➤ Spring boot 프로젝트 회원 가입 하기

- src -> main -> resources -> templates -> signInForm.html 작성한다.

```
<!DOCTYPE html>
<html lang="kor" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <form th:action="@{/member/signin_confirm}" method="POST">
        <input type="text" name="id" placeholder="id" />
        <br>
        <input type="password" name="pw" placeholder="pw" />
        <br>
        <input type="submit" value="SIGN-IN" />
    </form>
</body>
</html>
```



➤ Spring boot 프로젝트 회원 가입 하기

- src -> main -> java -> MemberController 클래스에서 회원가입 폼의 입력한 데이터를 클라이언트 signUpResult 페이지로 전달한다.

```
@PostMapping("/member/signup_confirm")
public String signUpConfirm(
    @RequestParam(value="id") String id,
    @RequestParam(value="pw") String pw,
    @RequestParam(value="email") String email, //요청파라미터
    Model model //데이터 전달
) {
    System.out.println("id : "+id);
    System.out.println("pw : "+pw);
    System.out.println("email : "+email);
    System.out.println("signUpComfirm");

    // 현재 시간 구하는 코드
    SimpleDateFormat sd = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
    Date now = new Date();

    //응답 model에 데이터를 넣어 클라이언트에 전달
    model.addAttribute("now", sd.format(now));
    model.addAttribute("new_member_id", id);

    return "signUpResult";
}
```



➤ Spring boot 프로젝트 회원 가입 하기

localhost:8090/member/signup

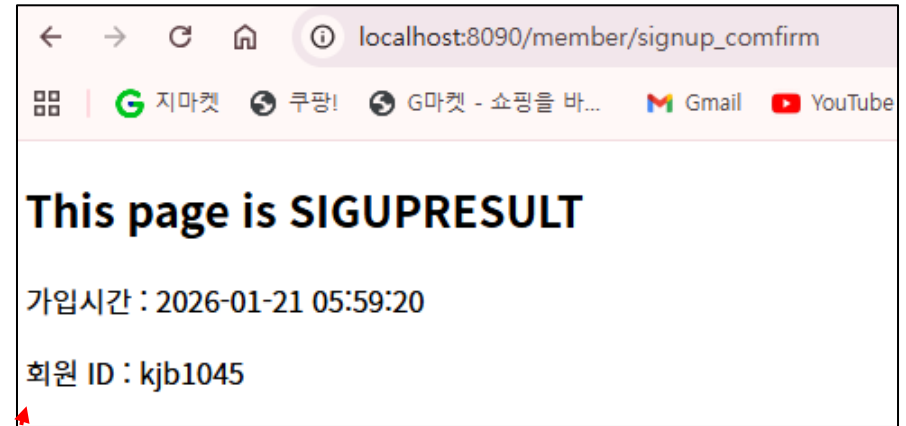
지마켓 쿠팡! G마켓 - 쇼핑을 바... Gmail

kjb1045

....

kjb1045@naver.com

SIGN-UP





➤ Spring boot 프로젝트 회원 가입 하기

- src -> main -> java -> MemberController 클래스에서 로그인폼에서 입력한 데이터를 클라이언트 signinResult 페이지로 전달한다.

```
@PostMapping("/member/signin_confirm")
public String signipConfirm(
    @RequestParam("id") String id,
    @RequestParam("pw") String pw,
    Model model //데이터 전달
) {
    System.out.println("id"+id);
    System.out.println("pw"+pw);
    System.out.println("signipComfirm");

    // 현재 시간 구하는 코드
    SimpleDateFormat sd = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
    Date now = new Date();

    model.addAttribute("now", sd.format(now));
    model.addAttribute("member_id", id);
    return "signinResult";
}
```



➤ Spring boot 프로젝트 회원 가입 하기

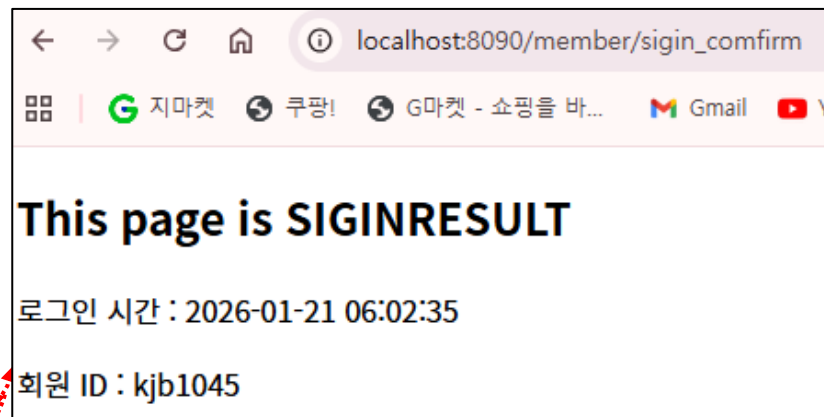
← → ↻ 🏠 ⓘ localhost:8090/member/singin

☰ | G 지마켓 쿠팡! G마켓 - 쇼핑을 바... Gmail

kjb1045

....

SING-IN



연습문제





1. 프로젝트 구조

- Controller는 `controller` 패키지에 생성
- HTML 파일은 `templates` 폴더의 하위 폴더에 생성

2. 필수 설정

- `application.properties`에 Thymeleaf 설정 확인
- Spring Boot 프로젝트 생성 시 Spring Web, Thymeleaf 의존성 추가



문제 1: 도서 대여 시스템

문제

도서관 도서 대여 시스템을 만드세요. 사용자가 도서명, 저자, ISBN, 대여자명을 입력하면 대여 정보를 보여주는 페이지로 이동합니다.

요구사항:

- /book/rental - 도서 대여 입력
- /book/rentalProc - 대여 정보 출력 페이지 필드:
- bookName, author, isbn, renterName

<출력 화면>

도서 대여 정보

도서명: 스프링 부트 완전정복

저자: 김영한

ISBN: 978-1234567890

대여자: 홍길동



문제 2: 상품 주문 시스템

문제

온라인 쇼핑몰의 상품 주문 시스템을 만드세요. 상품명, 가격, 수량, 주문자 정보를 입력받아 총 금액을 계산하여 보여줍니다.

요구사항:

- `/product/order` - 주문 입력 폼
- `/product/orderResult` - 주문 결과 페이지
- 필드: productName, price, quantity, ordererName
- 총 금액 = 가격 × 수량 계산

<출력 화면>

주문 정보

상품명: 무선 마우스

단가: 35000원

수량: 2개

주문자: 이영희

총 금액: 70000원



문제 3: 영화 예매 시스템

문제

영화관 예매 시스템을 만드세요. 영화 제목, 관람 날짜, 시간, 인원수, 예매자 정보를 입력받아 예매 확인 페이지를 보여줍니다.

요구사항:

- `/movie/reserve` - 예매 입력 폼
- `/movie/reserveConfirm` - 예매 확인 페이지
- 필드: movieTitle, reserveDate, reserveTime, people, reserverName

<출력 화면>

예매 확인
영화 제목: 어벤져스: 엔드게임
관람 날짜: 2026-01-25
관람 시간: 19:30
인원: 3명
예매자: 박민수
예매가 완료되었습니다!



문제 4: 레스토랑 예약 시스템

문제

레스토랑 예약 시스템을 만드세요. 예약자명, 전화번호, 예약 날짜, 예약 시간, 인원수를 입력받아 예약 완료 페이지를 보여줍니다.

요구사항:

- `/restaurant/booking` - 예약 입력 폼
- `/restaurant/bookingComplete` - 예약 완료 페이지
- 필드: `guestName`, `phone`, `bookingDate`, `bookingTime`, `partySize`

<출력 화면>

예약이 완료되었습니다
예약자명: 최지훈
연락처: 010-1234-5678
예약 날짜: 2026-01-30
예약 시간: 18:00
인원: 4명
예약 확인 문자가 발송되었습니다.



문제 5: 학원 수강 신청 시스템

문제

학원 수강 신청 시스템을 만드세요. 학생명, 연락처, 수강 과목, 수강 기간, 수강료를 입력받아 신청 완료 페이지를 보여줍니다.

요구사항:

- `/academy/enroll` - 수강 신청 폼
- `/academy/enrollResult` - 신청 완료 페이지
- 필드: `studentName`, `contact`, `subject`, `duration`, `tuition`

<출력 화면>

수강 신청 완료
학생명: 김수진
연락처: 010-9876-5432
수강 과목: 자바 프로그래밍
수강 기간: 6개월
수강료: 500000원
수강 신청이 완료되었습니다!
개강 안내 문자를 발송해 드리겠습니다.